

动态规划答题模板

1 动态规划基础

1.1 核心理想

将复杂问题分解为重叠子问题，通过保存子问题的解避免重复计算，自底向上或自顶向下（带记忆化）地构建原问题的解。

1.2 适用条件

1. 最优子结构：问题的最优解包含其子问题的最优解
2. 重叠子问题：递归算法会重复访问相同的子问题
3. 无后效性：当前状态确定后，不受之前决策的影响，只依赖当前状态

1.3 解题步骤

1. 定义状态：确定 dp 数组的含义
2. 状态转移：找出状态之间的关系方程
3. 初始条件：确定边界和初始值
4. 计算顺序：确定是自底向上还是自顶向下

1.4 与分治法的区别

- 分治法：各子问题独立，可能重复计算相同子问题
- 动态规划：子问题重叠，通过表格或记忆化存储中间结果
- 时间复杂度：分治可能重复计算，DP 通常更高效

2 背包问题系列

2.1 0-1 背包（每种物品只能选 0 个或 1 个）

状态： $dp[j]$ 表示容量为 j 时的最大价值。

转移方程：

$$dp[j] = \max(dp[j], dp[j - w[i]] + v[i])$$

实现：倒序遍历容量，确保每件物品只选一次。

```
1 for i in range(n):
2     for j in range(W, w[i]-1, -1):
3         dp[j] = max(dp[j], dp[j-w[i]] + v[i])
```

2.2 完全背包（每种物品无限供应）

状态： $dp[j]$ 表示容量为 j 时的最大价值。

转移方程：

$$dp[j] = \max(dp[j], dp[j - w[i]] + v[i])$$

实现：正序遍历容量，允许重复选择同一物品。

```
1 for i in range(n):
2     for j in range(w[i], W+1):
3         dp[j] = max(dp[j], dp[j-w[i]] + v[i])
```

2.3 多重背包（每种物品有数量限制 $cnt[i]$ ）

方法一：二进制拆分，将 $cnt[i]$ 拆分为 $1, 2, 4, \dots, 2^k$ ，剩余若干 0-1 背包物品。

方法二：单调队列优化，时间复杂度 $O(nW)$ 。

```
1 # 二进制拆分实现
2 for i in range(n):
3     k = 1
4     while k <= cnt[i]:
5         # 将k个物品i看作一个新物品
6         new_w = k * w[i]
7         new_v = k * v[i]
8         for j in range(W, new_w-1, -1):
9             dp[j] = max(dp[j], dp[j-new_w] + new_v)
10        cnt[i] -= k
11        k *= 2
12    # 处理剩余物品
13    if cnt[i] > 0:
14        for j in range(W, cnt[i]*w[i]-1, -1):
15            dp[j] = max(dp[j], dp[j-cnt[i]*w[i]] + cnt[i]*v[i])
```

2.4 混合背包（根据物品类型分类处理）

问题：包含 0-1、完全、多重三种类型物品。

思路：根据物品类型选择对应的转移方式。

```
1 for i in range(n):
2     if type[i] == 0: # 0-1背包：倒序
3         for j in range(W, w[i]-1, -1):
4             dp[j] = max(dp[j], dp[j-w[i]] + v[i])
5     elif type[i] == 1: # 完全背包：正序
6         for j in range(w[i], W+1):
7             dp[j] = max(dp[j], dp[j-w[i]] + v[i])
8     else: # 多重背包：二进制拆分
9         binary_split(i)
```

2.5 二维费用背包

状态： $dp[i][j]$ 表示第一维费用为 i 、第二维费用为 j 时的最大价值。

转移方程：

$$dp[i][j] = \max(dp[i][j], dp[i - v1[k]][j - v2[k]] + w[k])$$

实现：两层循环倒序。

```
1 for k in range(n):
2     for i in range(V1, v1[k]-1, -1):
3         for j in range(V2, v2[k]-1, -1):
4             dp[i][j] = max(dp[i][j],
5                             dp[i-v1[k]][j-v2[k]] + w[k])
```

2.6 分组背包

状态： $dp[j]$ 表示容量为 j 时的最大价值。

转移方程：

$$dp[j] = \max_{group} \max_{item \in group} (dp[j - w[item]] + v[item])$$

实现：对每组物品枚举，同一组内物品互斥。

```
1 for group in groups:
2     for j in range(W, -1, -1):
3         for item in group:
4             if j >= item.weight:
5                 dp[j] = max(dp[j],
6                             dp[j-item.weight] + item.value)
```

3 线性 DP 问题

3.1 数字三角形模型

问题：从三角形顶部到底部，求路径和的最大值（或最小值）。

状态： $dp[i][j]$ 表示到达位置 (i, j) 的最优值。

转移方程：

$$dp[i][j] = \max(dp[i-1][j-1], dp[i-1][j]) + a[i][j]$$

边界： $dp[1][1] = a[1][1]$ 。

空间优化：可倒序遍历，将二维压缩为一维。

```
1 # 从上到下
2 for i in range(2, n+1):
3     for j in range(1, i+1):
4         dp[i][j] = max(dp[i-1][j-1], dp[i-1][j]) + a[i][j]
5
6 # 空间优化版本
7 dp = [0] * (n+1)
8 for i in range(n):
9     for j in range(i, -1, -1): # 倒序更新
10        dp[j] = max(dp[j], dp[j-1]) + a[i][j]
```

3.2 最大子段和

问题：求连续子数组的最大和。

状态： $dp[i]$ 表示以 i 结尾的最大子段和。

转移方程：

$$dp[i] = \max(nums[i], dp[i-1] + nums[i])$$

结果： $\max(dp[i])$ 对所有 i 。

空间优化： $cur = \max(nums[i], cur + nums[i])$ 。

```
1 def max_subarray(nums):
2     if not nums: return 0
3     cur_sum = max_sum = nums[0]
4     for x in nums[1:]:
5         cur_sum = max(x, cur_sum + x)
6         max_sum = max(max_sum, cur_sum)
7     return max_sum
```

4 序列 DP 问题

4.1 最长递增子序列 (LIS)

问题：求最长严格递增子序列长度。

状态： $dp[i]$ 表示以 i 结尾的 LIS 长度。

转移方程：

$$dp[i] = \max(dp[j]) + 1, \forall j < i, nums[j] < nums[i]$$

解法一： $O(n^2)$ ，动态规划。

```
1 # O(n^2) 解法
2 def LIS_length(nums):
3     n = len(nums)
4     dp = [1] * n
5     for i in range(n):
```

```
6         for j in range(i):
7             if nums[j] < nums[i]:
8                 dp[i] = max(dp[i], dp[j] + 1)
9     return max(dp)
```

解法二： $O(n \log n)$ ，贪心 + 二分查找。

```
1 # O(n log n) 解法
2 from bisect import bisect_left
3
4 def LIS(nums):
5     tails = []
6     for x in nums:
7         i = bisect_left(tails, x)
8         if i == len(tails):
9             tails.append(x)
10        else:
11            tails[i] = x
12    return len(tails)
```

输出方案：记录每个位置的前驱，回溯输出。

4.2 最长公共子序列 (LCS)

问题：求两个序列的最长公共子序列（不要求连续）。

状态： $dp[i][j]$ 为 $text1[0 \dots i]$ 和 $text2[0 \dots j]$ 的 LCS 长度。

转移方程：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } text1[i] == text2[j] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{else} \end{cases} \quad (1)$$

```
1 def LCS(text1, text2):
2     m, n = len(text1), len(text2)
3     dp = [[0] * (n+1) for _ in range(m+1)]
4     for i in range(1, m+1):
5         for j in range(1, n+1):
6             if text1[i-1] == text2[j-1]:
7                 dp[i][j] = dp[i-1][j-1] + 1
8             else:
9                 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
10    return dp[m][n]
```

4.3 最长公共子串

问题：求两个序列的最长公共子串（要求连续）。

区别：要求连续，区别于 LCS。

状态： $dp[i][j]$ 表示以 $text1[i]$ 和 $text2[j]$ 结尾的最长公共子串长度。

转移方程：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } text1[i] == text2[j] \\ 0 & \text{else} \end{cases}$$

结果： $\max(dp[i][j])$ 对所有 i, j 。

```
1 def LCSSubstring(text1, text2):
2     m, n = len(text1), len(text2)
3     dp = [[0] * (n+1) for _ in range(m+1)]
4     max_len = 0
5     for i in range(1, m+1):
6         for j in range(1, n+1):
```

```

7         if text1[i-1] == text2[j-1]:
8             dp[i][j] = dp[i-1][j-1] + 1
9             max_len = max(max_len, dp[i][j])
10        else:
11            dp[i][j] = 0
12    return max_len

```

4.4 编辑距离 (Levenshtein 距离)

问题: 将 $word1$ 转换为 $word2$ 的最小操作次数 (插入、删除、替换)。

状态: $dp[i][j]$ 表示 $word1[0 \dots i]$ 转换为 $word2[0 \dots j]$ 的最小操作数。

转移方程:

$$dp[i][j] = \min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost) \quad (2)$$

其中 $cost = 0$ 当 $word1[i] == word2[j]$, 否则 $cost = 1$ 。

```

1 def minDistance(word1, word2):
2     m, n = len(word1), len(word2)
3     dp = [[0] * (n+1) for _ in range(m+1)]
4     for i in range(m+1):
5         dp[i][0] = i
6     for j in range(n+1):
7         dp[0][j] = j
8     for i in range(1, m+1):
9         for j in range(1, n+1):
10            if word1[i-1] == word2[j-1]:
11                dp[i][j] = dp[i-1][j-1]
12            else:
13                dp[i][j] = min(dp[i-1][j], dp[i][j-1],
14                               dp[i-1][j-1] + 1)
15    return dp[m][n]

```

5 区间 DP 问题

5.1 基础框架

特点: 先计算小区间, 再合并大区间。

状态: $dp[i][j]$ 表示区间 $[i, j]$ 的最优解。

转移方程:

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + w(i, j))$$

实现: 枚举分割点 k 。

```

1 for length in range(2, n+1): # 区间长度
2     for i in range(n-length+1):
3         j = i + length - 1
4         dp[i][j] = INF
5         for k in range(i, j):
6             dp[i][j] = min(dp[i][j],
7                             dp[i][k] + dp[k+1][j] + cost(i
8                             , j, k))

```

5.2 石子合并

问题: 合并相邻石子, 每次合并费用为石子总数, 求最小总费用。

状态: $dp[i][j]$ 表示合并区间 $[i, j]$ 的最小代价。

转移方程:

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j]) + sum[i][j]$$

其中 $sum[i][j]$ 为区间 $[i, j]$ 的石子总数 (前缀和计算)。

5.3 矩阵链乘

问题: 给定矩阵链, 求最优乘法顺序使计算量最小。

状态: $dp[i][j]$ 表示矩阵 $A_i \dots A_j$ 的最小计算量。

转移方程:

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + p[i-1] \times p[k] \times p[j])$$

其中 $p[i-1] \times p[k] \times p[j]$ 为最后一次乘法的计算量。

5.4 能量项链

问题: 环形区间 DP, 断环为链, 求最大能量。

转移方程:

$$dp[i][j] = \max_{i \leq k < j} (dp[i][k] + dp[k+1][j] + head[i] \times tail[k] \times tail[j])$$

实现: 将长度为 n 的环复制为长度 $2n$ 的链。

```

1 # 断环为链
2 n = len(heads)
3 extended_heads = heads + heads
4 extended_tails = tails + tails
5
6 for length in range(2, n+1):
7     for i in range(2*n - length + 1):
8         j = i + length - 1
9         for k in range(i, j):
10            dp[i][j] = max(dp[i][j],
11                           dp[i][k] + dp[k+1][j] +
12                           extended_heads[i] * extended_tails[k] *
13                           extended_tails[j])

```

6 树形 DP 问题

6.1 基本模型

树的直径: 树上最短路径的最长路径。

方法: 维护最大深度和次大深度, $diameter = \max_d1 + \max_d2$ 。

```

1 def tree_diameter(root):
2     max_diameter = 0
3
4     def dfs(u, parent):
5         nonlocal max_diameter
6         max1 = max2 = 0 # 最大和次大深度
7         for v in graph[u]:
8             if v == parent: continue
9             depth = dfs(v, u) + 1
10            if depth > max1:
11                max2, max1 = max1, depth
12            elif depth > max2:
13                max2 = depth
14            max_diameter = max(max_diameter, max1 +
15                               max2)
16        return max1
17
18    dfs(root, -1)
19    return max_diameter

```

最大独立集：树上选点，相邻节点不能同时选。

状态：

$dp[u][0]$: 不选 u , $\sum \max(dp[v][0], dp[v][1])$

$dp[u][1]$: 选 u , $\sum dp[v][0] + 1$

转移方程：

$$\begin{cases} dp[u][0] = \sum_{v \in children(u)} \max(dp[v][0], dp[v][1]) \\ dp[u][1] = \sum_{v \in children(u)} dp[v][0] + 1 \end{cases} \quad (3)$$

```
1 def max_independent_set(root):
2     def dfs(u, parent):
3         dp0 = dp1 = 0
4         for v in graph[u]:
5             if v == parent: continue
6             dfs(v, u)
7             dp0 += max(dp[v][0], dp[v][1])
8             dp1 += dp[v][0]
9         dp[u][0] = dp0
10        dp[u][1] = dp1 + 1
11
12    dfs(root, -1)
13    return max(dp[root][0], dp[root][1])
```

6.2 换根 DP (二次扫描)

问题：求以每个节点为根时的 DP 值。

思路：

- 第一次 DFS：计算子树贡献（自底向上）。
- 第二次 DFS：计算父节点方向贡献（自顶向下，换根）。

```
1 def rerooting_dp():
2     # 第一次DFS: 计算子树贡献
3     def dfs1(u, p):
4         for v in graph[u]:
5             if v != p:
6                 dfs1(v, u)
7                 dp[u] += dp[v] + size[v]
8
9     # 第二次DFS: 换根, 计算父节点方向贡献
10    def dfs2(u, p):
11        for v in graph[u]:
12            if v != p:
13                # 从u换根到v
14                dp[u] -= dp[v] + size[v]
15                size[u] -= size[v]
16                size[v] += size[u]
17                dp[v] += dp[u] + size[u]
18
19        dfs2(v, u)
20
21    # 回溯, 恢复状态
22    dp[v] -= dp[u] + size[u]
23    size[v] -= size[u]
24    size[u] += size[v]
25    dp[u] += dp[v] + size[v]
```

6.3 树上背包

问题：在树上选若干节点，容量限制下求最大价值。

状态： $dp[u][j]$ 表示以 u 为根的子树中选 j 个节点的最大价值。

转移：对每个子树进行分组背包。

```
1 def dfs(u, p):
2     dp[u][0] = 0
3     for v in children[u]:
4         if v == p: continue
5         dfs(v, u)
6         # 倒序枚举容量 (分组背包)
7         for j in range(m, -1, -1):
8             for k in range(j):
9                 dp[u][j] = max(dp[u][j],
10                                dp[u][j-k] + dp[v][k])
```

7 状态压缩 DP

7.1 旅行商问题 (TSP)

状态： $dp[mask][i]$ ：经过点集 $mask$ ，当前在点 i 的最短路径。

```
1 for mask in range(1 << n):
2     for i in range(n):
3         if (mask >> i) & 1:
4             prev = mask ^ (1 << i)
5             for j in range(n):
6                 if (prev >> j) & 1:
7                     dp[mask][i] = min(dp[mask][i],
8                                         dp[prev][j] + dist[j][i])
```

7.2 骨牌铺设

状态：按列进行状态压缩，轮廓线 DP。

7.3 骑士放置问题

问题：在 $n \times n$ 棋盘上放置骑士，互不攻击。

状态： $dp[i][mask]$ 表示前 i 行，当前行状态为 $mask$ 的最大骑士数。

转移：枚举当前行状态，检查与上一行是否冲突。

```
1 def solve():
2     for i in range(1, n+1):
3         for mask in valid_masks:
4             for prev in valid_masks:
5                 if not conflict(mask, prev):
6                     dp[i][mask] = max(dp[i][mask],
7                                         dp[i-1][prev] +
8                                         count(mask))
```

8 数位 DP

核心：按位搜索，使用 ‘memo’ 记录状态。

```
1 def dfs(pos, tight, lead, state):
2     if pos == -1: return 1
3     if not tight and not lead and memo[state]:
4         return memo[state]
5     limit = digits[pos] if tight else 9
6     ans = 0
7     for i in range(limit + 1):
8         # 状态更新逻辑...
9         ans += dfs(pos-1, ..., new_state)
10    return ans
```

9 概率与期望 DP

特点：通常倒序推导（从终点到起点）。

方程： $E[i] = \sum (E[next] \times P(i \rightarrow next)) + cost$ 。

10 DP 优化技巧

10.1 单调队列优化

适用于 $dp[i] = \max(dp[j] + val[j]) + C$ 形式，其中 j 有滑动窗口限制。

维护一个单调递减（或递增）的 ‘deque’ 存储下标。

10.2 斜率优化

适用于形如 $dp[i] = \max / \min(dp[j] + (i - j)^2)$ 或类似带平方项的转移。

核心思想：将 j 视为变量， i 视为常数，将转移方程转化为 $y = kx + b$ 形式。

步骤：

1. 展开方程，分离 i 和 j
2. 形如 $dp[i] - i^2 = dp[j] + j^2 + 2ij$
3. 令 $Y = dp[j] + j^2$, $X = j$, $K = -2i$, $B = dp[i] - i^2$
4. 维护下凸包（求最大值）或上凸包（求最小值）

实现：用双端队列维护凸包，每次加入新点时剔除无用点。

10.3 四边形不等式优化

适用于区间 DP: $dp[i][j] = \min(dp[i][k] + dp[k+1][j]) + w(i, j)$ 。

若 w 满足四边形不等式，则最优分割点 $s[i][j]$ 满足：

$$s[i][j-1] \leq s[i][j] \leq s[i+1][j]$$

可将复杂度从 $O(n^3)$ 降至 $O(n^2)$ 。